M1 explanation script

MEMBER M1 / SCENARIOS S1 + S2

Use the short version for the group's eight-minute demonstration. The longer version is a study walkthrough or individual defense, not an extra eight minutes to add to the group video. Say "synthetic" when describing the supplied experiments. After you run the ROS/video tests, replace the final limitations sentence with your actual results.

Short presentation version - approximately two minutes

My part is M1: passenger detection and demand prediction. It covers scenarios S1 and S2.

First, the detector estimates how many people are currently waiting in the queue. It uses track IDs so repeated observations do not count the same person multiple times. Queue membership uses each person's ground position, and a short confirmation delay reduces flickering. The system also preserves an identity across a brief occlusion.

The detector publishes queue state five times per second. It reports the count, confidence, validity, and whether the pedestrian crossing is occupied. If observations stop arriving, it marks the old count invalid instead of treating missing data as an empty queue.

The predictor samples queue history once per second. After collecting sixty seconds, it uses the current count, recent average, trend, and estimated increases and decreases to predict the queue thirty seconds ahead. I compare ridge regression with a persistence baseline, which simply predicts that the count stays the same.

The supplied experiment uses thirty synthetic six-minute episodes, split by whole episode into training, validation and test sets. On the held-out data, ridge achieved an average error of 2.89 people, compared with 4.02 for persistence.

I also tested the simple detector on one hundred real Mall dataset images. Its average error was 25.51 people, showing that crowded real scenes need a stronger detector. My output goes to M2's coordinator; calibrated queue-video and full ROS integration still need verification.

Detailed explanation - task and purpose

My responsibility is to turn observations of passengers into two useful pieces of information: the current queue state and a forecast of future demand. The current state helps the hub understand what is happening now. The forecast helps the coordinator decide whether transport service may be needed soon.

These are different problems. Detection asks, "How many people are waiting now?" Prediction asks, "How many people may be waiting thirty seconds from now?" My work covers both, while M2 owns bus dispatch and M5 owns the independent safety response.

The assignment uses a twelve-by-eight-metre map. The queue occupies x from two to six and y from five to seven. The crossing is a separate region. A person can be in the crossing without being in the queue, so I calculate those states independently.

Detailed explanation - project structure

The project separates algorithms from ROS communication. This lets me test most logic without launching a simulator.

The file `core.py` contains the polygon membership test, passenger track state, freshness checks, crossing hold, feature extraction and prediction history. `scenario.py` generates the repeatable synthetic observations. `nodes.py` connects that logic to ROS publishers, subscribers and timers. `vision.py` supplies the optional camera baseline: person detection, conversion to map coordinates, and association of detections between frames.

The experiment script generates data, trains the model and saves evaluation evidence. The test file checks counting, occlusion, invalid input, timing and feature behavior. The data, models and evidence directories make the results reproducible rather than leaving them only in presentation slides.

Detailed explanation - S1 detection and tracking

The detector supports two input paths. For deterministic testing, it receives tracks with known identities and map positions. For the camera path, the supplied baseline uses OpenCV's HOG person detector and a simple nearest-position tracker. The camera path is included but has not been executed in this environment.

For a video detection, the bottom centre of the person's bounding box approximates where their feet meet the ground. A homography maps that image point into map coordinates. It must be calibrated using at least four noncollinear points on the floor. A fabricated camera calibration would give meaningless map positions, so actual measured pixel coordinates are required.

The queue count is the number of confirmed tracks inside the queue polygon. It is not the total number of people ever seen, and it is not simply the number of detection boxes in a frame.

A new track needs about zero point four seconds of evidence before confirmation. Queue-boundary changes are also debounced. A zero point three second occlusion can retain the existing identity because the tracker tolerates a short absence of up to half a second. Longer disappearance removes the old track rather than preserving a passenger indefinitely.

The deterministic test begins with zero passengers, adds three at five seconds, adds two more at ten seconds and removes one at fifteen seconds. After the confirmation delay, the counts are zero, three, five and four. Repeating an ID in the observations does not increase the count.

Detailed explanation - validity and the crossing

The output includes validity because a missing observation is different from observing zero people.

I preserve the source observation timestamp. Republishing the same old observation does not refresh it. I also track local receipt age with a monotonic clock, so a frozen source cannot appear healthy just because a timer continues running. If the source or receipt is too old, the queue becomes invalid. Future and out-of-order timestamps are checked as well.

In the synthetic freeze test, observations stop at thirty seconds. The first invalid state appears at thirty-one seconds, which is within the assignment's one point two second requirement.

Crossing occupancy is conservative. A crossing detection blocks clearance immediately, even before a queue track would finish confirmation. Once the pedestrian leaves, the crossing must be continuously observed clear for two seconds. Missing or invalid input cannot prove clearance. My output informs the coordinator and safety supervisor; it does not directly command a vehicle to move or stop.

Detailed explanation - S2 features and model

The predictor consumes queue messages and samples once per second. It maintains sixty consecutive seconds of valid history. During warm-up, missing input or a timing gap, its forecast is invalid and history must be collected again.

The feature vector contains five numbers: the current count; the mean of the latest ten counts; the change since ten seconds earlier; the sum of positive count changes over thirty seconds; and the sum of negative count changes over thirty seconds.

The last two are count-change proxies. They are not exact passenger arrivals and departures. If two people arrive and two leave at the same moment, the total count may stay unchanged. The supplied QueueState interface does not include entry and exit events, so this limitation is documented rather than pretending those flows are known.

The simple baseline is persistence: predict that the queue thirty seconds from now will equal its current count. The trained model is ridge regression, which learns a weighted combination of the features with regularization to discourage unnecessarily large coefficients. Negative forecasts are clamped to zero. A fractional forecast represents expected demand; it is not a fractional detected person.

Detailed explanation - data split and leakage prevention

The experiment generates thirty independent synthetic episodes, each lasting six minutes. Eighteen episodes are used for training, six for validation and six for the final test.

I split whole episodes before extracting overlapping windows. Otherwise, nearly identical windows from one episode could appear in both training and test data and make the evaluation look better than it really is.

Feature normalization is fitted using training data only. Regularization strength is chosen on validation episodes. The final test episodes are used to report performance, not to choose the model. Future queue values are labels for training and evaluation; they are never online input features. The hidden future arrival and boarding schedule is used only by the synthetic generator.

The model selection procedure also compares validation performance with persistence. If persistence is better, the saved deployment policy selects it. This does not guarantee that the selected method wins on every later episode.

Detailed explanation - results and honest interpretation

The packaged evaluation contains one thousand six hundred thirty-two forecast windows across six held-out episodes. Ridge has mean absolute error 2.89 people and root mean square error 3.75 people. Persistence has mean absolute error 4.02 and root mean square error 5.29.

That means the trained model improves average forecast accuracy on this synthetic test set. It does not establish the same accuracy on real passengers. The windows overlap within each episode, so they are

not one thousand six hundred thirty-two independent experiments.

The model also loses on some episodes. For example, on seed 1027, ridge's average error is about 1.79, while persistence is about 1.12. Keeping that failure visible helps explain when a simple baseline can be useful.

Fourteen portable unit tests passed. The supplied model-call timing is well below the hundred-millisecond target on this computer, but it measures only the regression call. It excludes camera detection, feature preparation, ROS communication and full system response. It must not be presented as end-to-end latency.

Detailed explanation - the real dataset and failed baseline

I added the original Mall dataset from its university source. It contains two thousand real images with manual head-position and crowd-count labels. I evaluated one hundred images chosen by a fixed, evenly spaced sampling rule, using the supplied HOG baseline without tuning it on those images.

The average absolute count error was twenty-five point five one people. That is a poor result: many people in the image are small, far away or partly hidden, and the simple full-body detector misses them. The result is kept in the report as a failed baseline and a reason to improve perception. It does not cancel the passing synthetic logic tests; it shows that correct tracking logic and accurate real-image detection are different requirements.

The output includes annotated images, with publisher head positions and detected boxes, so the failure can be inspected. These are the dataset publisher's manual labels, not a claim that I personally annotated every frame. The dataset is for noncommercial research and must be cited.

The mall benchmark counts people in the publisher's image region. It does not supply the assignment's map calibration or actual bus-stop queue. The published frame cadence is not precise enough to silently assume a thirty-second timeline, and it is not thirty independent demand episodes. Therefore I use this real-data result as additional perception evidence and retain the required synthetic prediction experiments.

Detailed explanation - handoff and remaining work

The detector publishes `/hub/queue` at a nominal five hertz. The predictor subscribes to it and publishes `/hub/demand` at a nominal one hertz. These are topics because they carry repeated state updates; neither operation needs a request-and-response service.

M2's coordinator uses the forecast to decide whether to request a bus through an action. The specified rule requires three consecutive valid forecasts of at least six, an available bus, one outstanding request at a time and a thirty-second cooldown. It clears escalation after five forecasts below four. When forecasting is unavailable, M2 can use a clearly labeled fallback based on fresh observed queue data. Those action and dispatch checks belong to the integration with M2.

For my required action-lifecycle explanation, I can trace how demand causes a DockBus goal, how acceptance differs from completion, how feedback reports progress, and how cancellation or failure ends the mission. I do not claim to implement the bus action server.

Before final submission, I need a clean ROS build, measured publication rates, compatible QoS and recorded topic evidence. I also need calibrated real video with at least one hundred manually labeled frames, including occlusion. M2 and M5 must verify the bus response, crossing inhibition and paired

waiting-time comparison. Prediction accuracy by itself does not prove that passengers waited less.

My independent review responsibility is M3's state validity. I should check that stale parking or traffic data becomes UNKNOWN rather than falsely free, and document the actual review finding once their code is available.

Likely defense questions and answers

Why track IDs instead of counting boxes? Boxes can repeat across frames or overlap around the same person. IDs represent a continuing person and let me retain identity through a short occlusion.

Why use the foot-point? It approximates a person's position on the calibrated ground plane. The image box centre is above that plane and gives a poorer ground-position estimate.

Why not report zero when the camera freezes? Zero is a valid observation of an empty queue. A frozen feed provides no current evidence, so validity must be false.

Why sixty seconds if some features use thirty? The assignment requires a sixty-second history. My implementation waits for that full history even though the selected features need shorter recent slices.

Why ridge regression? It is small, interpretable, quick to evaluate and appropriate as a documented baseline beyond persistence. More complex models are not automatically better on limited data.

Can the count-change features measure every arrival? No. Simultaneous entry and exit can cancel in the count. Accurate flow features would require additional track transition events.

What is MAE? The average absolute difference between predicted and true future queue count, measured in people. RMSE penalizes large errors more strongly.

How do you avoid leakage? Split entire episodes first, fit scaling only on training data, choose settings only on validation data and reserve test episodes for the final report.

Does lower MAE mean shorter waits? No. Compare reactive and predictive dispatch on identical episodes, record boarding times and include passengers still waiting at the end.

What repaired failure can you explain? The tests protect against a frozen source remaining valid when its message is republished. Preserve source time and refuse to refresh the receipt watchdog for duplicate stamps. Demonstrate the passing regression test; do not claim a historical bug or personal implementation experience you have not actually had.

What is incomplete? The real Mall image baseline was measured and performs poorly. Calibrated queue-video accuracy, ROS runtime execution, integrated action dispatch and crossing-stop evidence remain unverified.

Why not replace all synthetic data with Mall? The assignment explicitly requires independent demand episodes and deterministic safety/fault scenarios. Mall provides real image labels, but not the same episode, timing, queue or crossing contracts. Use it as supplementary evidence with attribution.